



Department of Engineering Cybernetics

TTK4147 Real-time Systems

08 More Scheduling

08.10.2025

1

TTK4147 – 2025: More Scheduling



1

Lecture and exercise plan 2025

Date	Week	Lectures Wednesday 10:15 - 13:00 in H1- Main Building
2025-08-20	34	01 Introduction
2025-08-27	35	02 Hardware and operation systems
2025-09-03	36	03 Multiprogramming and Memory Management
2025-09-10	37	04 Scheduling
2025-09-17	38	05 Deadlocks, mixed criticality and freertos
2025-09-24	39	06 Priority Inversion
2025-10-01	40	07 Linux
2025-10-08	41	08 More Scheduling
2025-10-15	42	9 Guest lecture
2025-10-22	43	10 QNX
2025-10-29	44	11 Repetition
2025-11-05	45	12 System Architectures
2025-11-12	46	13 Analysis of Physical systems
2025-11-19	47	14 Repetition

2025-11-24 51 Exam 15:00 - 19:00

Date	Week	Exercises Monday 14:00 - 18:00, Friday 14:00 - 18:00, Sanntidslabben, G203/G204
	34	
2025-08-25 / 2025-08-22	35	0- Basics and setup
2025-09-01 / 2025-09-05	36	1 - Time
2025-09-08 / 2025-09-12	37	2 - Memory
2025-09-15 / 2025-09-19	38	3 - Scheduling
2025-09-22 / 2025-09-26	39	4 - Concurrency
2025-09-29 / 2025-10-03	40	5 - AVR UC3
2025-10-06 / 2025-10-10	41	6 - AVR FreeRTOS
2025-10-13 / 2025-10-17	42	7 - BRTT and Rpi
2025-10-20 / 2025-10-24	43	8 - BRTT and PREEMPT RT
2025-10-29	44	Theory Exercise, in Sentralbygg 1 S3 at 18:00 (Starts 18:15)
	45	
	46	

2

TTK4147 – 2025: More Scheduling



2

Today

- Deadline Monotonic Priority Ordering
- Sporadic and Aperiodic Tasks
 - How to Schedule
 - Polling Server
 - Deferable Server
 - Sporadic Server
- Extended Task Model
- Earliest Deadline First
- Multiprocessor Scheduling
 - Multicore computer
 - Effect on real-time

3

TTK4147 – 2025: More Scheduling



3

Response Time Analysis Repetition, Fixed Priority Scheduling

(Calculating worst-case response time for each task, check that all WCRTs ≤ periods)

$$R_i = C_i + I_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

Number of Releases = $\left\lceil \frac{R_i}{T_j} \right\rceil$ (= Number of times task i is interrupted) Rounded up

$\left\lceil \frac{R_i}{T_j} \right\rceil C_j$ (= The time task j takes from task i)

$$w_i^{n+1} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil C_j$$

Task	Period (T)	Time (C)	Priority (P)	Utilization (U)	Response time (R)
1	7	3	3	42.9%	
2	12	2	2	16.7%	
3	20	5	1	25.0%	
			Sum	84.6%	

$$w_1^0 = C_1 = 3 \implies r_1 = 3$$

4

TTK4147 – 2025: More Scheduling



4

Response Time Analysis Repetition

Task	Period (T)	Time (C)	Priority (P)	Utilization (U)	Response time (R)
1	7	3	3	42.9%	3
2	12	2	2	16.7%	
3	20	5	1	25.0%	
Sum				84.6%	

$$w_2^0 = C2 = 2$$

$$w_2^1 = C2 + \left\lceil \frac{w_2^0}{T1} \right\rceil C1 = 2 + \left\lceil \frac{2}{7} \right\rceil 3 = 5$$

$$w_2^2 = C2 + \left\lceil \frac{w_2^1}{T1} \right\rceil C1 = 2 + \left\lceil \frac{5}{7} \right\rceil 3 = 5 \Rightarrow r_2 = 5$$

5

TTK4147 – 2025: More Scheduling

NTNU

5

Response Time Analysis Repetition

Task	Period (T)	Time (C)	Priority (P)	Utilization (U)	Response time (R)
1	7	3	3	42.9%	3
2	12	2	2	16.7%	5
3	20	5	1	25.0%	18
Sum				84.6%	



$$w_3^0 = C3 = 5$$

$$w_3^1 = C3 + \left\lceil \frac{w_3^0}{T1} \right\rceil C1 + \left\lceil \frac{w_3^0}{T2} \right\rceil C2 = 5 + \left\lceil \frac{5}{7} \right\rceil 3 + \left\lceil \frac{5}{12} \right\rceil 2 = 10$$

$$w_3^2 = 5 + \left\lceil \frac{10}{7} \right\rceil 3 + \left\lceil \frac{10}{12} \right\rceil 2 = 13$$

$$w_3^3 = 5 + \left\lceil \frac{13}{7} \right\rceil 3 + \left\lceil \frac{13}{12} \right\rceil 2 = 15$$

$$w_3^4 = 5 + \left\lceil \frac{15}{7} \right\rceil 3 + \left\lceil \frac{15}{12} \right\rceil 2 = 18$$

$$w_3^5 = 5 + \left\lceil \frac{18}{7} \right\rceil 3 + \left\lceil \frac{18}{12} \right\rceil 2 = 18 \Rightarrow r_3 = 18$$

6

TTK4147 – 2025: More Scheduling

NTNU

6

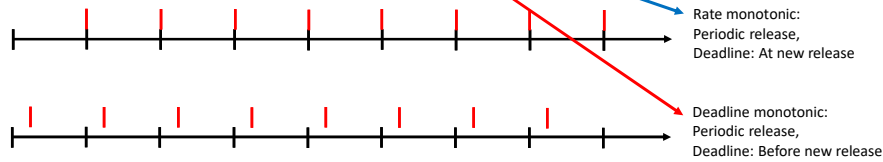
Today's topic: Deadline Monotonic Priority Ordering

- Rate monotonic priority is not suitable when deadline is shorter than period ($D < T$).
Why?
- Deadline monotonic priorities on deadline (D) instead of period (T).

Task	Period (T)	Time (C)	Priority (P)
1	7	3	3
2	12	2	2
3	20	5	1

- NOT THE SAME as Earliest deadline first, which will be described later today.
- If $D = T$, then deadline monotonic and rate monotonic are the same.

Task	Period (T)	Deadline (D)	Time (C)	Priority (P)
A	20	5	3	4
B	15	7	3	3
C	10	10	4	2
D	20	20	3	1



7

TTK4147 – 2025: More Scheduling

NTNU

Deadline Monotonic Example, worst-case time analysis

Task	Period (T)	Deadline (D)	Time (C)	Priority (P)
A	20	5	3	4
B	15	7	3	3
C	10	10	4	2
D	20	20	3	1

$$w_i^{n+1} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil C_j$$

$$\omega_A^0 = C_A = 3$$

$$\omega_B^0 = C_B = 3$$

$$\omega_B^1 = C_B + \left\lceil \frac{\omega_B^0}{T_A} \right\rceil C_A = 3 + \left\lceil \frac{3}{20} \right\rceil 3 = 6$$

$$\omega_B^2 = C_B + \left\lceil \frac{\omega_B^1}{T_A} \right\rceil C_A = 3 + \left\lceil \frac{6}{20} \right\rceil 3 = 6$$

$$\omega_C^0 = C_C = 4$$

$$\omega_C^1 = C_C + \left\lceil \frac{\omega_C^0}{T_A} \right\rceil C_A + \left\lceil \frac{\omega_C^0}{T_B} \right\rceil C_B$$

$$\omega_C^1 = 4 + \left\lceil \frac{4}{20} \right\rceil 3 + \left\lceil \frac{4}{15} \right\rceil 3 = 10$$

$$\omega_C^2 = C_C + \left\lceil \frac{\omega_C^1}{T_A} \right\rceil C_A + \left\lceil \frac{\omega_C^1}{T_B} \right\rceil C_B$$

$$\omega_C^2 = 4 + \left\lceil \frac{10}{20} \right\rceil 3 + \left\lceil \frac{10}{15} \right\rceil 3 = 10$$

8

TTK4147 – 2025: More Scheduling

NTNU

8

Deadline Monotonic Example

Task	Period (T)	Deadline (D)	Time (C)	Priority (P)	wcet
					Response time (R)
A	20	5	3	4	3
B	15	7	3	3	6
C	10	10	4	2	10
D	20	20	3	1	20



$$\begin{aligned}\omega_D^0 &= C_D = 3 \\ \omega_D^1 &= C_D + \left\lceil \frac{\omega_D^0}{T_A} \right\rceil C_A + \left\lceil \frac{\omega_D^0}{T_B} \right\rceil C_B + \left\lceil \frac{\omega_D^0}{T_C} \right\rceil C_C \\ \omega_D^1 &= 3 + \left\lceil \frac{3}{20} \right\rceil 3 + \left\lceil \frac{3}{15} \right\rceil 3 + \left\lceil \frac{3}{10} \right\rceil 4 = 13 \\ \omega_D^2 &= C_D + \left\lceil \frac{\omega_D^1}{T_A} \right\rceil C_A + \left\lceil \frac{\omega_D^1}{T_B} \right\rceil C_B + \left\lceil \frac{\omega_D^1}{T_C} \right\rceil C_C \\ \omega_D^2 &= 3 + \left\lceil \frac{13}{20} \right\rceil 3 + \left\lceil \frac{13}{15} \right\rceil 3 + \left\lceil \frac{13}{10} \right\rceil 4 = 17\end{aligned}$$

$$\begin{aligned}\omega_D^3 &= C_D + \left\lceil \frac{\omega_D^2}{T_A} \right\rceil C_A + \left\lceil \frac{\omega_D^2}{T_B} \right\rceil C_B + \left\lceil \frac{\omega_D^2}{T_C} \right\rceil C_C \\ \omega_D^3 &= 3 + \left\lceil \frac{17}{20} \right\rceil 3 + \left\lceil \frac{17}{15} \right\rceil 3 + \left\lceil \frac{17}{10} \right\rceil 4 = 20 \\ \omega_D^4 &= C_D + \left\lceil \frac{\omega_D^3}{T_A} \right\rceil C_A + \left\lceil \frac{\omega_D^3}{T_B} \right\rceil C_B + \left\lceil \frac{\omega_D^3}{T_C} \right\rceil C_C \\ \omega_D^4 &= 3 + \left\lceil \frac{20}{20} \right\rceil 3 + \left\lceil \frac{20}{15} \right\rceil 3 + \left\lceil \frac{20}{10} \right\rceil 4 = 20\end{aligned}$$

9

TTK4147 – 2025: More Scheduling

NTNU

9

Sporadic and Aperiodic Tasks

- Aperiodic tasks can be released at any time.
 - Period (T) can be represented as the average release interval.
- Sporadic tasks are aperiodic tasks that have a minimum interval between releases.
 - Period (T) can be represented as the minimum or average release interval.
- The simple task model assumes that the deadline of the task should complete before its next release (max D = T).
- Occur infrequently, but often urgent when they do. (faults, alarms, ...)
- Can still use response time analysis.
- Sporadic and Aperiodic release of tasks mirrors the nature of many real world events
- How to ensure real-time characteristic / predictability of Sporadic and Aperiodic tasks?

10

TTK4147 – 2025: More Scheduling

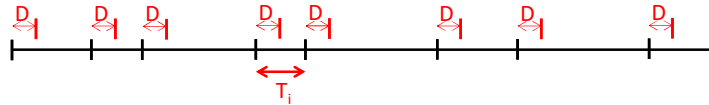
NTNU

10

How to Schedule Sporadic and Aperiodic Tasks

For sporadic tasks, deadline monotonic is often suitable.

- Sporadic task is modeled as a task with period equal to the minimum (or average) release time.



- Deadline is the sporadic tasks actual deadline.

11

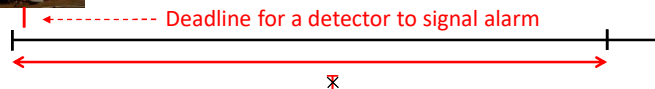
TTK4147 – 2025: More Scheduling

NTNU

11

How to Schedule Sporadic and Aperiodic Tasks *cont.*

- Not suitable to use period equal to the minimum release time for aperiodic tasks, since we do not have a defined minimum release time.



- In a rate/deadline monotonic scheme, one could give aperiodic tasks the lowest priority (?).
 - It will only run when the system otherwise is idle.
 - Can suffer from starvation.
 - What about alarms, faults, ...?

12

TTK4147 – 2025: More Scheduling

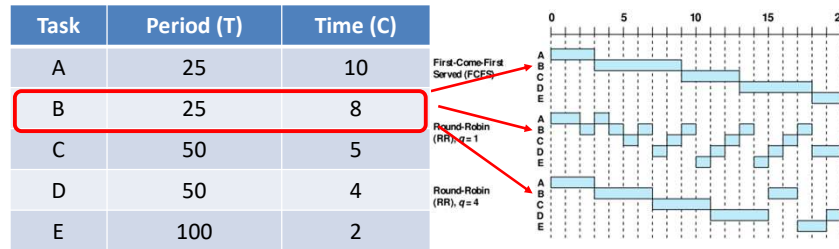
NTNU

12

Execution-time Servers

To provide better responses for aperiodic tasks, we can use Execution-time Servers.

- Runs as one task / process in the system.



- May have a high priority
- Maintain a budget for *clients*, typically aperiodic tasks.

13

TTK4147 – 2025: More Scheduling



Execution-time Servers continue

We will investigate 3 different Execution-time Servers:

- Periodic/Polling Server (PS)
- Deferrable Server (DS)
- Sporadic Server (SS)

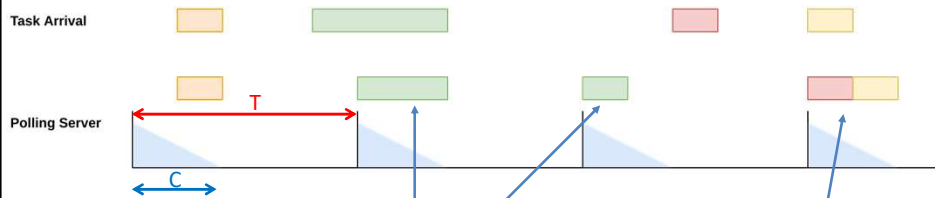
14

TTK4147 – 2025: More Scheduling



Periodic/Polling Server (PS)

- Runs periodically, with a time budget.
 - Period = T , Budget = C
 - Can be scheduled by rate-monotonic together with the other tasks in the system



- Each period, a the computation time budget is available for any pending clients.
 - If a client require more work than available within a budget, then it will have to continue next period.
 - If more than one clients are pending, they have to be scheduled somehow within the server.
- The budget has to be used when the server is running or it is wasted.

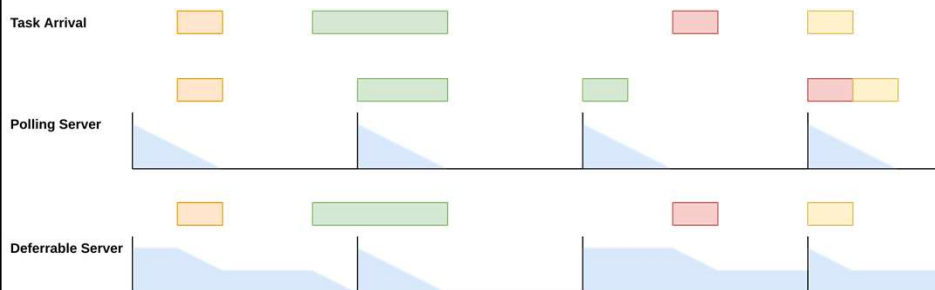
15

TTK4147 – 2025: More Scheduling

NTNU

Deferrable Server (DS)

- Differs from polling server in situations where budget and CPU are available but no client.
 - Deferrable server only lose budget when used.



- Deferrable server have better response time for aperiodic tasks.
- Back-to-back, where one budget is used just after another.



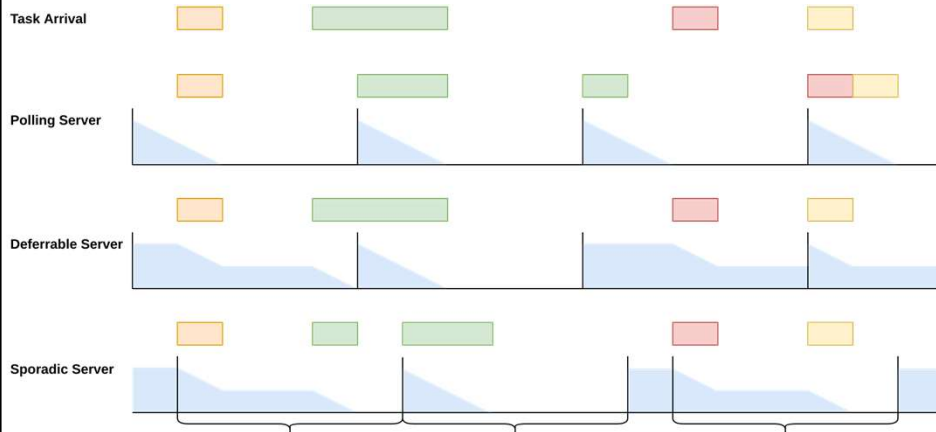
16

TTK4147 – 2025: More Scheduling

NTNU

Sporadic Server (SS)

- Differs from deferred server on its replenish policy.
 - Budget will be replenished one time period after it has started to be used.
 - A period starts when someone starts using the budget



17

TTK4147 – 2025: More Scheduling

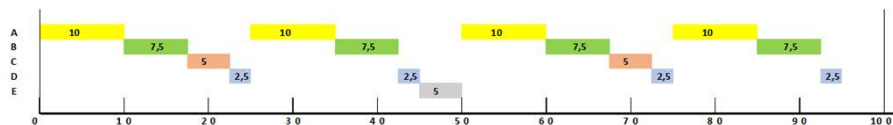
NTNU

17

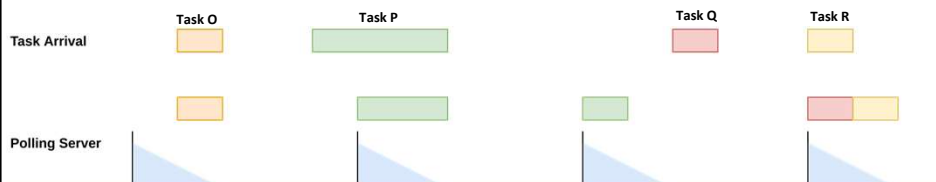
Execution-time Servers, Runs as one task / process in the system, Polling Server

- May have a high priority

Task	Period(T)	Time (c)	Priority	Utilization
A	25	10	5	0,4
B	25	7,5	4	0,3
C	50	5	3	0,1
D	50	5	2	0,1
E	100	5	1	0,05
				0,95



- Maintain a budget for *clients*, typically aperiodic tasks.



18

TTK4147 – 2025: More Scheduling

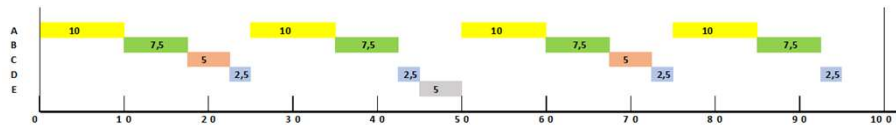
NTNU

18

Execution-time Servers, Runs as one task / process in the system, Deferrable Server

- May have a high priority

Task	Period(T)	Time (c)	Priority	Utilization
A	25	10	5	0,4
B	25	7,5	4	0,3
C	50	5	3	0,1
D	50	5	2	0,1
E	100	5	1	0,05
				0,95



- Maintain a budget for *clients*, typically aperiodic tasks.



19

TTK4147 – 2025: More Scheduling

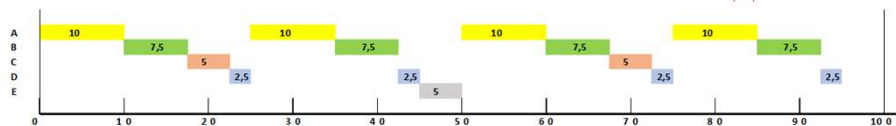


Execution-time Servers, Runs as one task / process in the system, Sporadic Server

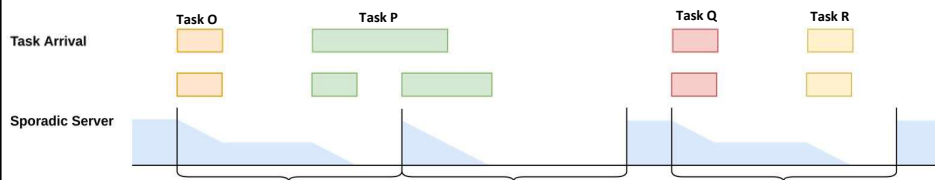
- May have a high priority

Task	Period(T)	Time (c)	Priority	Utilization
A	25	10	5	0,4
B	25	7,5	4	0,3
C	50	5	3	0,1
D	50	5	2	0,1
E	100	5	1	0,05
				0,95

5 tasks, 0,95 utilization???



- Maintain a budget for *clients*, typically aperiodic tasks.



20

TTK4147 – 2025: More Scheduling



Complete Model

- In order to have an accurate model of execution / response time*, you can add expressions for:
 - Context switches between the tasks.
 - Interrupt handling when an aperiodic or sporadic task arrives (handling is high priority even if task is not).
 - Cost of scheduling itself.

*Response-time: The time from a task starts execution to it is completed
[Burns and Wellings]

21

TTK4147 – 2025: More Scheduling



21

An Extendible Task Model

Starting point: Simple task model:

- | | |
|---|--|
| - Fixed set of tasks | - Periodic tasks, known periods |
| - Independent tasks | - No system overheads |
| - Deadlines equal to periods | - Fixed worst-case execution times |
| - No internal delays or blocking requests | - All tasks on a single processor / core |

22

TTK4147 – 2025: More Scheduling



22

An Extendible Task Model continue

So far:

- Deadlines can be less than the period ($D < T$)
- Sporadic and aperiodic tasks, as well as periodic tasks, can be supported
- Task interactions are possible, with the resulting blocking being factored into the response time equations

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

$I_i =$ **Interference**

$$B_i = \sum_{k=1}^K usage(k, i) C(k)$$

Blocking (once pr. shared resources used by lower and higher or equal (than i) priority tasks.)

B_i is the maximum blocking time that task i can suffer.

$usage$ is a 0/1 function:

$usage(k, i) = 1$ if resource k is used by at least one task with a priority less than P_i , and at least one task with a priority greater or equal to P_i . Otherwise, it gives the result 0.

$C(k)$ is the worst-case execution time of the k critical section.

23

TTK4147 – 2025: More Scheduling



23

Further model extensions

- Release Jitter
- Arbitrary Deadlines
- Cooperative Scheduling
- Fault Tolerance
- Offsets
- Optimal Priority Assignment

(and combinations)

24

TTK4147 – 2025: More Scheduling



24

Release Jitter

The maximum variation in a task's release is called its **release jitter, J**.

Example:

Release of a task from the end of another task with varying response time

J influence on the time a task can be effected by other tasks and thus, on the response time.

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_j + J_j}{T_j} \right\rceil C_j$$

25

TTK4147 – 2025: More Scheduling

NTNU

25

Arbitrary Deadlines

What if D_i (and hence potentially R_i) is $> T_i$?

In the following we assume that the release of a task will be delayed until any previous releases of the same task have completed.

More (2, 3, ... q) succeeding releases may have $R > T$



26

TTK4147 – 2025: More Scheduling

NTNU

26

Arbitrary Deadlines continue

Worst case response time for all (foregoing q + current), $q = 0, 1, 2, \dots$:

$$w_i^{n+1}(q) = B_i + (q + 1)C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n(q)}{T_j} \right\rceil C_j$$

Worst case response time for q 'te element:

$$R_i(q) = w_i^n(q) - qT_i$$

The number of releases that need to be considered is bounded by the lowest value of q for which the following relation is true

$$R_i(q) \leq T_i$$



The worst-case response time is then the maximum value found for each q :

$$R_i = \max_{q=0,1,2,\dots} R_i(q)$$

27

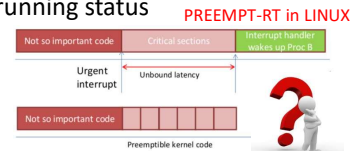
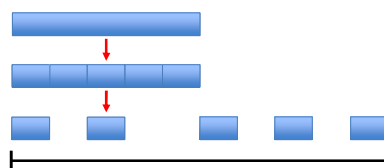
TTK4147 – 2025: More Scheduling

NTNU

27

Cooperative Scheduling

- True preemptive behavior is not always acceptable for safety-critical systems
- Cooperative scheduling or deferred preemption splits tasks into non-preemptive slots, voluntarily “yields” its running status



What is the difference?

In Linux kernel the critical sections are decreased, i.e. where it cannot be interrupted, while in cooperative the task yields its running status

- Mutual exclusion is via non-preemption

28

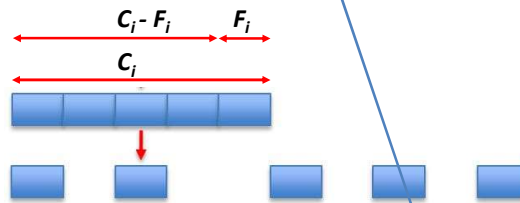
TTK4147 – 2025: More Scheduling

NTNU

28

Cooperative Scheduling continue

- The use of deferred preemption has two important advantages
 - It increases the schedulability of the system, and it can lead to lower values of R
 - With deferred preemption, no interference can occur during the last slot of execution



- Let the execution time of the final block be F_i

$$w_i^{n+1} = B_{MAX} + C_i - F_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil C_j$$

$$R_i = w_i^n + F_i$$



What is the gain of this?

The time for when it can be interfered is shorter

29

TTK4147 – 2025: More Scheduling

NTNU

29

Fault Tolerance

- Fault tolerance via either forward or backward error recovery always results in extra computation
- In a real-time fault tolerant system, deadlines should still be met even when a certain level of faults occur
- If F is the number of faults allowed during one execution

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j + \max_{k \in hp(i)} FC_k^f$$

C_i^f is the extra computation time that results from an error in task i.
 $hp(i)$ is set of tasks with priority higher or equal (hep) to i.

- If there is a minimum arrival interval T_f for faults

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j + \max_{k \in hp(i)} \left(\left\lceil \frac{R_i}{T_f} \right\rceil C_k^f \right)$$

30

TTK4147 – 2025: More Scheduling

NTNU

30

Earliest Deadline First (EDF)

- Not as important or in use as Fixed Priority Scheduling (FPS), thus it will not be covered as much.
- But, you should know what it is, and how it can be analyzed.
- Firstly, it is NOT the same as deadline monotonic, so do not confuse the two.

31

TTK4147 – 2025: More Scheduling



31

Earliest Deadline First (EDF) Scheduling

- The task that has the shortest time to deadline will always be running.
 - The priority of each task will change dynamically based on the current deadlines.
 - Schedulability can also be tested with utilization, but with another formula than for Fixed Priority Scheduling (FPS).
 - Just as utilization for FPS, it also require simple task model.
- | | |
|---|------------------------------------|
| - Fixed set of tasks | - Periodic tasks, known periods |
| - Independent tasks | - No system overheads |
| - Deadlines equal to periods | - Fixed worst-case execution times |
| - No internal delays or blocking requests | - All tasks on a single processor |

32

TTK4147 – 2025: More Scheduling



32

Earliest Deadline First (EDF) utilization

- Utilization, given a simple task model.

$$\sum_{i=1}^N \frac{C_i}{T_i} \leq 1$$

- This means that EDF is able to schedule any task set as long as it takes less time to execute than what is available.
 - In this sense we can say that EDF is optimal.

Utilization for Fixed Priority Scheduling $U \equiv \sum_{i=1}^N \frac{C_i}{T_i} \leq N(2^{1/N} - 1)$

33

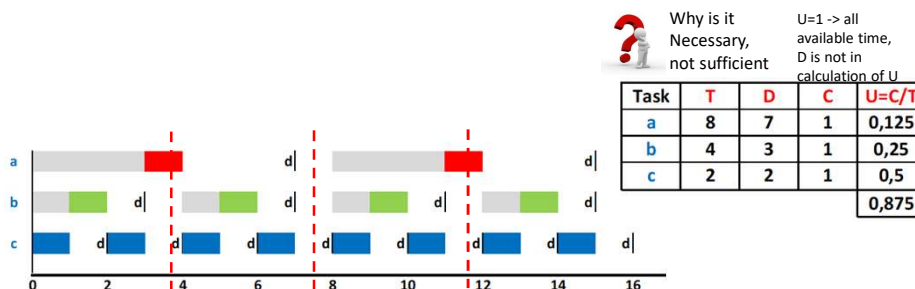
TTK4147 – 2025: More Scheduling

NTNU

33

Earliest Deadline First **Without Simple Task Model**

- How can we analyze EDF when the task model is not simple (which basically is all actual systems)? (Given $D < T$)
 - For FPS, we can use response time analysis, which can incorporate delays from shared resources, task switching delays etc.
 - Response time analysis can be used for EDF, but will be much more complicated.



The question is:
At any time t , is the foregoing deadlines met?

$$\sum_{i=1}^N \frac{C_i}{T_i} \leq 1 \quad \text{still has to be fulfilled}$$

34

TTK4147 – 2025: More Scheduling

NTNU

34

How to judge schedulability at EDF, using PDC?

1. Processor Demand Criteria (PDC)

- Assume that system starts at $t = 0$.
 - All tasks are released at this point.
 - All tasks will arrive at their maximum frequency at any future time.

(Necessary,
not sufficient)

Task	T	D	C	U=C/T
a	8	7	1	0,125
b	4	3	1	0,25
c	2	2	1	0,5
				0,875

- It is then possible to calculate all future deadlines.
 - Then evaluate each deadline to check that they are met.

$$h(t) = \sum_{i=1}^N \left\lfloor \frac{t + T_i - D_i}{T_i} \right\rfloor C_i$$

$h(t)$: The amount of work that must be completed before t , i.e. all jobs that had deadlines before (or at) t .

Requirement:

$$\forall t > 0, h(t) \leq t$$

for t in each deadline

Load at time t is less than or equal to t , for all t (given that all deadlines are met)

$\forall$ means for all

35

TTK4147 – 2025: More Scheduling

NTNU

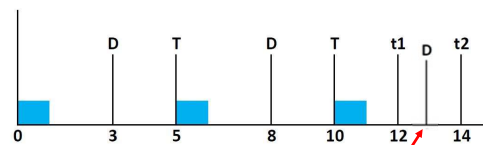
35

Example; Schedulability at EDF, using PDC

Single task: $T = 5, D = 3, C = 1$

Investigate two values of t :

$$t_1 = 12, t_2 = 14$$



$$h(t) = \sum_{i=1}^N \left\lfloor \frac{t + T_i - D_i}{T_i} \right\rfloor C_i$$

$$\forall t > 0, h(t) \leq t$$

$$t_1 = 12: t + T - D = 14,$$

$$h(12) = 2 \leq 12 \quad (2 \text{ because has not yet reached } D \text{ at } t=13)$$

$$\left\lfloor \frac{12 + 5 - 3}{5} \right\rfloor \cdot 1 = 2$$



$$t_2 = 14: t + T - D = 16,$$

$$h(14) = 3 \leq 14$$



36

TTK4147 – 2025: More Scheduling

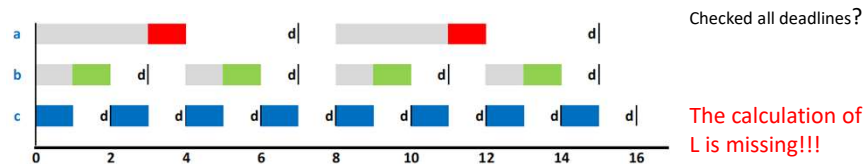
NTNU

36

How to judge schedulability, using PDC continue

Problem is to know which t to check.

- In rate and deadline monotonic, we can release all tasks at $t=0$, and know that this is the worst possible starting point.
For EDF, we do not know this.
- Only values of t that correspond to deadlines of tasks need be checked;
- Necessary to calculate a value L , and if we reach $t = L$ without any deadline missed, then the system is schedulable.
- For larger task sets, L can be very large $\rightarrow$ many deadlines / t to check.



37

TTK4147 – 2025: More Scheduling



37

How to calculate L

Two ways of calculating L .

$$1. L_a = \left[\sum_{i=1}^N ((T_i - D_i) * (C_i / T_i)) \right] / (1 - U).$$

$$2. L_b = \omega^j \text{ for } \omega^j = \omega^{j+1}$$

$$w^0 = \sum_{i=1}^N C_i$$

$$w^{j+1} = \sum_{i=1}^N \left\lceil \frac{w^j}{T_i} \right\rceil C_i$$

$$L = \min(L_a, L_b)$$

Task	T	D	C	U=C/T
a	8	7	1	0,125
b	4	3	1	0,25
c	2	2	1	0,5
				0,875

38

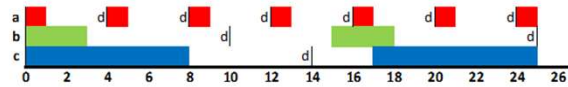
TTK4147 – 2025: More Scheduling



38

Example, how to calculate L

Task	T	D	C	U	La
a	4	4	1	0,25	0,00
b	15	10	3	0,20	12,59
c	17	14	8	0,47	17,78
			0,92		30,37



$$1. L_a = [\text{sum}_{i=1 \rightarrow N} ((T_i - D_i) * (C_i / T_i))] / (1 - U).$$

$$2. L_b = \omega^j \text{ for } \omega^j = \omega^{j+1}$$

$$w^0 = \sum_{i=1}^N C_i \quad w^{j+1} = \sum_{i=1}^N \left\lceil \frac{w^j}{T_i} \right\rceil C_i$$

	a	b	c	
w0	1	3	8	12
w1	3	3	8	14
w2	4	3	8	15
w3	4	3	8	15

$$L = \min(L_a, L_b) = \min(30.37, 15) = 15$$

→ Check all deadlines for $t \leq 15$

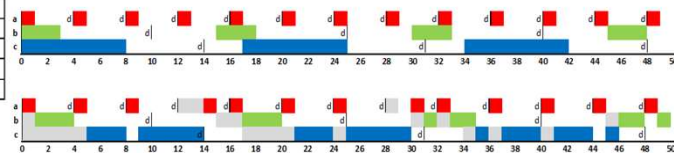
39

TTK4147 – 2025: More Scheduling

NTNU

Example, how to calculate L → Check schedulability

Task	T	D	C	U
a	4	4	1	0,25
b	15	10	3	0,20
c	17	14	8	0,47
			0,92	



Check all deadlines for $t \leq 15$: 4, 8, 10, 12, 14

t	a	b	c	h(t)
15	3	3	8	14
14	3	3	8	14
12	3	3	0	6
10	2	3	0	5
8	2	0	0	2
4	1	0	0	1

$$h(t) = \sum_{i=1}^N \left\lceil \frac{t + T_i - D_i}{T_i} \right\rceil C_i$$

$$\forall t > 0, h(t) \leq t$$



40

TTK4147 – 2025: More Scheduling

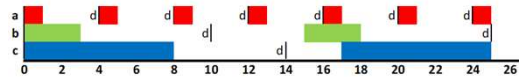
NTNU

How to judge schedulability at EDF, using QPA?

2. Quick Processor-demand Analysis (QPA)

is a faster way to do this analysis, as number of deadlines to check is reduced.:

- Calculate L as in PDC



- Starts at $t = L$ and works backwards.

- Use the same formula and criterion

- Only checks some critical deadlines.

$$h(t) = \sum_{i=1}^N \left\lfloor \frac{t + T_i - D_i}{T_i} \right\rfloor C_i \quad \forall t > 0, h(t) \leq t$$

- $h(L) = s$
- If $s > L \rightarrow$ unschedulable, if not $\rightarrow$ don't need to check for $t: s < t < L$
- Repeat for $h(h(L)) = h(s)$
- What if $h(s) = s$?



Use the largest deadline having $D < s$

t	a	b	c	h(t)
15	3	3	8	14
14	3	3	8	14
12	3	3	0	6
6	1	0	0	1
1	0	0	0	0

- Can have as little as 1% the effort as PDC, but equally correct.

41

TTK4147 – 2025: More Scheduling

NTNU

41

Fixed Priority vs Earliest Deadline First

- EDF is guaranteed to work for higher utilization.
- EDF is more difficult to implement, and will have more overhead. (Dynamic priority)
- It is more difficult to incorporate tasks without deadlines in EDF.
- FPS is more predictable if the system gets overloaded.
 - You know that it is the lowest priority task that will fail its deadlines.
- In EDF you can get a cascade effect, where the fact that one task fail its deadline will cause multiple others to do the same.
- Systems scheduable in FPS is also scheduable in EDF.
Not necessary the other way around
- EDF gets more out of the processor, can have higher utilization than FPS



What (EDF or FPS) would you use in a mixed criticality system? Why?

42

TTK4147 – 2025: More Scheduling

NTNU

42

Multiprocessor Scheduling

- How to schedule on modern CPUs that have multiple cores?
- Probably material for a course by its own, will only cover some basic concept and how this affect real-time
- We will only consider multicore computers.
 - Multiple CPU cores on the same physical CPU chip.
 - Share main memory.
 - Usually share operating system.
- We are not considering three other types of multiprocessor systems mentioned in the book.
 - Computers with multiple CPUs (although it will in practice be the same if they are sharing memory)
 - Clusters of computers
 - I/O processors

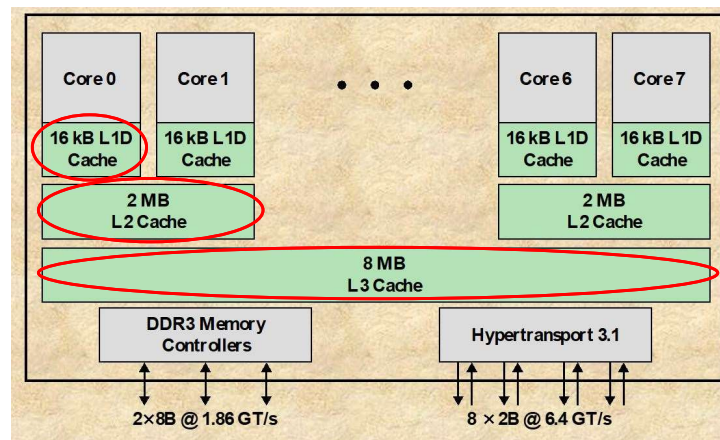
43

TTK4147 – 2025: More Scheduling

NTNU

43

Multicore CPU



Want to minimize access to off-chip memory instead of to maximize processor utilization



If allocating one core to each thread, how should threads in the same process be allocated?

Threads / processes sharing memory should be in adjacent cores

44

TTK4147 – 2025: More Scheduling

NTNU

44

Assignment of Processes to Cores

Three strategies to be considered:

1. Static assignment of processes to a core.

- Each process is permanently assigned to a core.
- One queue of processes for each core.
- Minimal scheduling overhead.
- This can result in one core being idle, while another is very busy.

2. A global queue for dynamic assignment of processes to a core.

- Each process is added to a global queue of processes.
- When a processor core is idle, it will select the next process and start running it.
- No core is idle as long as there is available work to do.
- Preempted or blocked processes are likely to run on a different core when activated next time, which makes caching less efficient.

3. Dynamic load balancing.

- Each core has a queue of processes.
- A process can be moved between core queues depending on load (which makes caching less efficient).

45

TTK4147 – 2025: More Scheduling



45

How to Assign Processes

Two architectures are focused:

1. Master/slave architecture

- Kernel scheduling functions (part of OS) always run on the same *master* core.
- *Slave* cores sends requests for services to the master (with the OS)
- Simple and requires little changes from uniprocessor scheduling.
- Master can become a bottleneck and a single point of failure.

2. Peer architecture

- Kernel scheduling functions can run on any core.
- Each core schedule itself with available processes from a pool.
- Must make sure that not two cores run the same process.

Also variations between these two extremes.

46

TTK4147 – 2025: More Scheduling



46

Multiprogramming on Individual Processors?

- Seems like a stupid question, why shouldn't you?

Not only one answer, but usually:

- If you have many cores, it might not be important that all are utilized fully.
- If a process can get full access of a processor core of its own, many issues become simpler.
 - Basically what some OS (VxWorks) provide with CPU reservation.
 - Can be used in real-time, by giving important processes its own core.

47

TTK4147 – 2025: More Scheduling

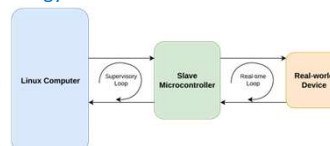


47

Real-time on Multicore

- Real-time and multicore is difficult, since you introduce another potential uncertainty (and by that weaken the predictability).
 - To move a thread from one core to another will cause a longer (and more unpredictable) delay than normal scheduling.
 - Should at least be able to know that a thread runs on a specific core.
(May use static assignment)
- But it can have beneficial applications:
 - Run hard real-time tasks (all or some) on a dedicated core.
 - Run all non-real-time tasks on a separate core.

Strategy 3 for Linux RT: Slave Microcontroller



48

TTK4147 – 2025: More Scheduling



48



Also use

Blackboard → TTK4147 Sanntidssystemer (2025 HØST) →
Learning materials → Questions

49

TTK4147 – 2025: More Scheduling



49

Fredrik Böttger Bratberg
Theory Exercise 2019 - task 9
Collapse

The solution two this question has made me confused about the details of execution time servers.

In the solution it appears that the server never actually runs? For example in time period 3, b runs instead of the server even though the servers priority is way higher. I undertand that there are no pending clients but the server need runtime to actually check/poll if there are clients, no?

I hope we can clear up how the server is scheduled together with other tasks in the sytem. I think the slides and the book does not cover this properly.

4. Execution time servers

Two periodic tasks and an execution time server are running on a system using fixed-priority scheduling, where 1 is the lowest priority. The execution time server is a *polling server* (as opposed to a *deferrable* or *sporadic* server), with a replenishment period of 3 time units and a time budget of 1 time unit. The two tasks and the server are all released at time 0.

Task	Period	Execution time	Priority
a	12	3	1
b	8	4	2
server	3	1	99

Two aperiodic tasks (that only run once each) are to be run on the execution time server:

Task	Release time	Execution time
t	7	4
u	20	1

Task 9

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
a	↑																								
b	↑																								
s	↑																								
t																									
u																									

1p: t and u (server) only run for 1 time-unit each server period (can give point for deferrable server)

1p: Uses polling server (t and u runs only on 9, 12, 15, 18, 21 and budget is refilled periodically)

1p: All tasks meet deadlines

1p: Matches figure

This is partly a question of notation. An arrow up means that a task is or should be released. With respect to the server, which has the highest priority, it will always start when it is released, but if there is no possible task to run under the server (and with the server's priority), the server will back out and leave the time slot to other tasks. If it is a task to run under the server, this task will start immediately. As the budget is only 1, the rest of the task will run (partly) at the next release of the server. Due to the notation, the task running under the server will have its own timeline, but running with the server's priority.

50

TTK4147 – 2025: More Scheduling



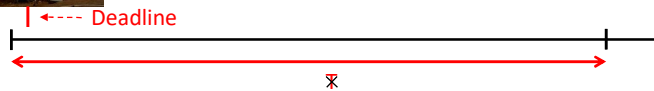
50

Exercise

What is an aperiodic task, and why would you not give these fixed priorities in the same way they are assigned to periodic tasks?

Not suitable to use period equal to the minimum release time for aperiodic tasks, since we do not have a defined minimum release time.

→ Could give aperiodic tasks the lowest priority.



51

TTK4147 – 2025: More Scheduling

NTNU

51

Exercise

2018-7

What is an aperiodic task, and how would you schedule it in a system running other periodic tasks?

Task is performed when an event occur. No limitations for when the event can occur. To provide better responses for aperiodic tasks, we can use an Execution-time Servers.

52

TTK4147 – 2025: More Scheduling

NTNU

52

Exercise

2023-10

- 1) How do preemptive and cooperative scheduling differ in their approach?
- 2) What makes cooperative scheduling less suitable for hard real-time applications?

1)

Preemptive:

Interrupts on time(out). Time slice.

Cooperative:

Running task triggers task switch. Yield. Run-to-completion. Splits tasks into slots/sub-tasks. Triggers on (arrival of higher) priority.

2)

Unknown when the running task will yield. Unknown amount of interference.

Cannot ensure deadline. Cannot ensure response time.

Is unfair. Priority inversion (high priority tasks spend more time blocked).

53

TTK4147 – 2025: More Scheduling

NTNU

53

Exercise

2021-15

What is true about dynamic load balancing?

Select one or more alternatives:

1. Each process is added to a global queue of processes. 
2. Each core has a queue of processes. 
3. When a processor core is idle, it will select the next process and start running it. 
4. No core is idle as long as there is available work to do. 
5. It can result in one core being idle, while another is very busy. 
6. A process can be moved between core queues depending on load. 

54

TTK4147 – 2025: More Scheduling

NTNU

54